Statistical and Machine Learning

Exemplar and Kernel Methods — Teacher Version

Instructions

All questions should first be discussed in pairs or small groups. Afterwards, we shall discuss them together.

Exercise 1 – k-Nearest Neighbours: Classification and Regression

(a) A k-NN classifier estimates the class probabilities for a new point $\mathbf{x}$ using its k nearest neighbours $N_{\mathbf{x}}^k$:

$$\hat{P}(Y = l \mid \mathbf{x}) = \frac{1}{k} \sum_{i \in N_{\mathbf{x}}^k} \mathbf{1}_{\{y_i = l\}}.$$

Suppose $k = 5$ and the five nearest neighbours of a query point have labels $(+, +, -, +, -)$. Compute $\hat{P}(Y = + \mid \mathbf{x})$ and state the predicted class.

(b) For k-NN regression the prediction is

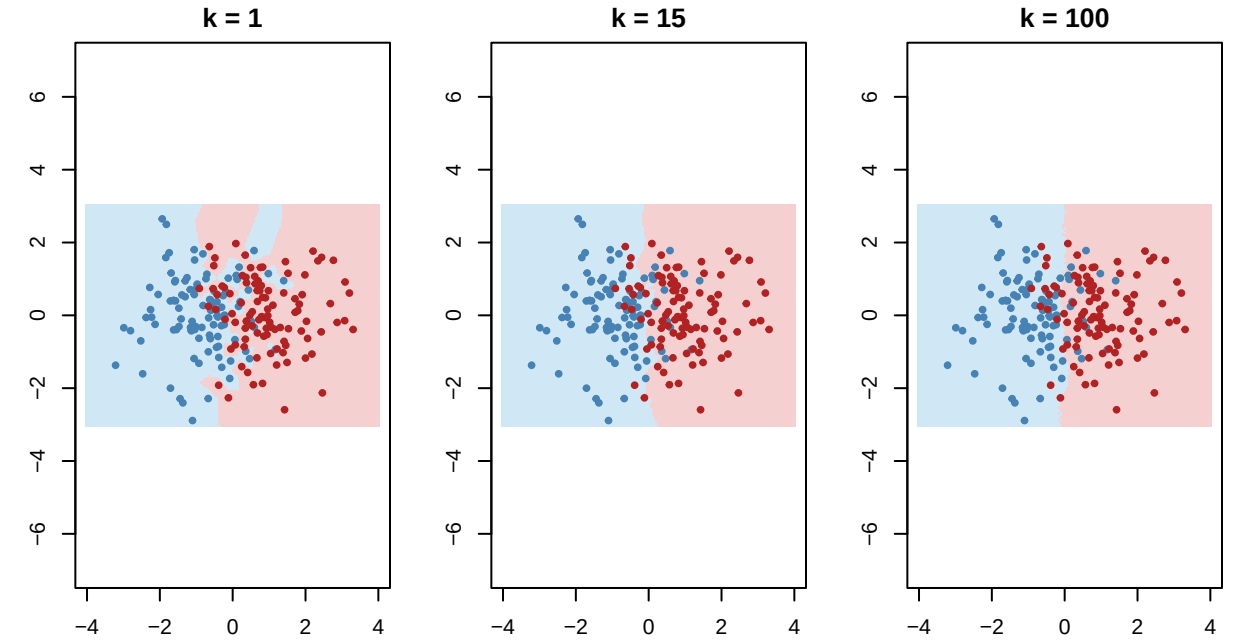
$$\hat{f}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_{\mathbf{x}}^k} y_i.$$

Using the same five neighbours as in (a), now with continuous responses $y = (3.2,\ 4.1,\ 1.0,\ 3.8,\ 2.5)$, compute $\hat{f}(\mathbf{x})$.

(c) The code below generates a binary classification dataset and fits k-NN for three values of k. Run the code, examine the decision boundaries, and answer: which value of k leads to under-fitting, which to over-fitting, and which provides the best trade-off? Explain how k controls the bias–variance trade-off in k-NN.

```
library(class)

set.seed(1)
n     <- 200
x1    <- c(rnorm(n/2, -1), rnorm(n/2,  1))
x2    <- c(rnorm(n/2,  0), rnorm(n/2,  0))
y     <- factor(rep(c("A", "B"), each = n/2))
train <- data.frame(x1 = x1, x2 = x2, y = y)

# Prediction grid
grid  <- expand.grid(x1 = seq(-4, 4, length.out = 150),
                     x2 = seq(-3, 3, length.out = 150))

par(mfrow = c(1, 3), mar = c(3, 3, 2, 1))
for (k_val in c(1, 15, 100)) {
  pred <- knn(train[, 1:2], grid, train$y, k = k_val)
  plot(grid$x1, grid$x2, col = ifelse(pred == "A", "#d0e8f5", "#f5d0d0"),
       pch = 15, cex = 0.4,
```

```
       main = paste0("k = ", k_val),
       xlab = "x1", ylab = "x2", asp = 1)
  points(x1, x2, col = ifelse(y == "A", "steelblue", "firebrick"),
         pch = 19, cex = 0.6)
}
```




```
par(mfrow = c(1, 1))
```

(d) State one advantage and one disadvantage of k-NN compared to a parametric classifier such as logistic regression.

Solution

(a) Three of the five neighbours are class +, so $\hat{P}(Y = + \mid \mathbf{x}) = 3/5 = 0.60$. Since $0.60 > 0.5$, the predicted class is +.

(b)

$$\hat{f}(\mathbf{x}) = \frac{3.2 + 4.1 + 1.0 + 3.8 + 2.5}{5} = \frac{14.6}{5} = 2.92.$$

(c) $k = 1$ over-fits: the decision boundary follows every training point, giving zero training error but high test variance. $k = 100$ under-fits: so many neighbours are averaged that the boundary becomes almost linear, ignoring local structure (high bias). $k = 15$ is the best trade-off for this dataset: the boundary is smooth enough to generalise but flexible enough to capture the non-linear structure.

In general, smaller $k \Rightarrow$ lower bias, higher variance (the model is sensitive to individual training points). Larger $k \Rightarrow$ higher bias, lower variance (predictions are averaged over many neighbours and are more stable but may miss local patterns). The optimal k is usually selected by cross-validation.

(d) Advantage: k-NN is fully non-parametric; it makes no assumptions about the shape of the decision boundary and can outperform parametric classifiers when the true boundary is highly non-linear.

Disadvantage: k-NN does not provide a coefficient table, so it gives no indication of which predictors are important. It also scales poorly with large n and p (storing and searching through all training observations at prediction time).

Exercise 2 — Kernel Density Estimation and Bandwidth Selection

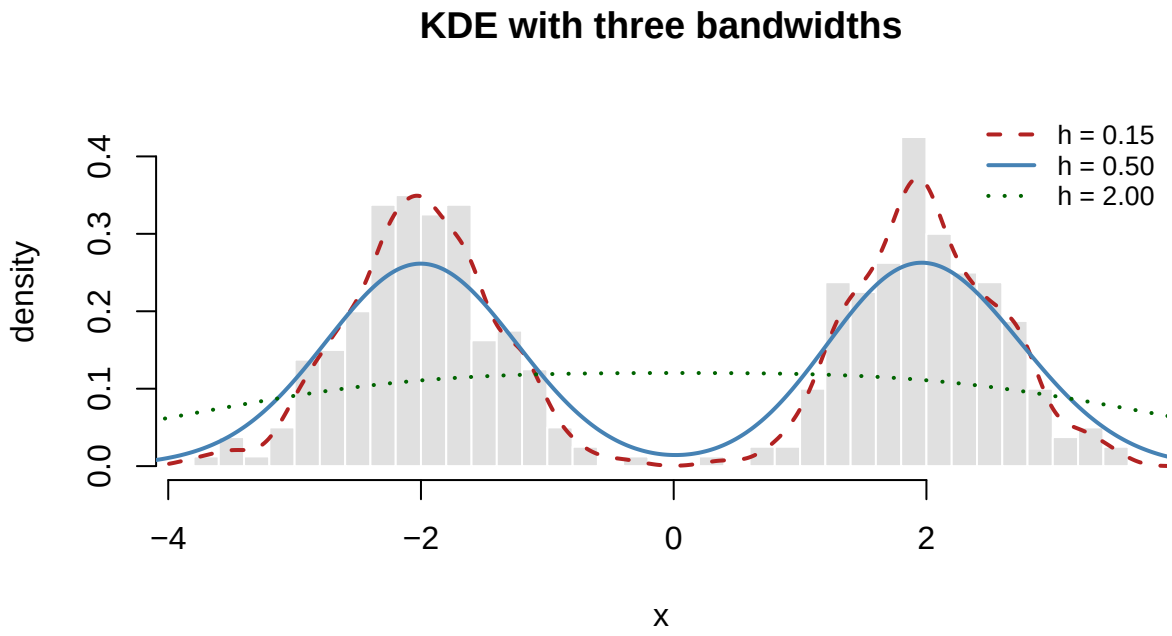
(a) A univariate kernel density estimator (KDE) is defined as

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - X_i}{h}\right),$$

where K is a kernel function and $h > 0$ is the bandwidth. State the three properties a function K must satisfy to be a valid kernel (density) function, and verify that the Gaussian kernel $K(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$ satisfies them.

(b) The code below generates data from a mixture of two Gaussians and fits KDEs with three different bandwidths. Run the code, then describe the effect of each bandwidth on the estimated density.

```r
set.seed(42)
x <- c(rnorm(200, mean = -2, sd = 0.6),
       rnorm(200, mean =  2, sd = 0.6))

d_small  <- density(x, bw = 0.15)
d_medium <- density(x, bw = 0.50)
d_large  <- density(x, bw = 2.00)

hist(x, breaks = 30, probability = TRUE,
     col = "grey88", border = "white",
     main = "KDE with three bandwidths",
     xlab = "x", ylab = "density",
     ylim = c(0, 0.45))
lines(d_small,  col = "firebrick",   lwd = 2, lty = 2)
lines(d_medium, col = "steelblue",   lwd = 2, lty = 1)
lines(d_large,  col = "darkgreen",   lwd = 2, lty = 3)
legend("topright",
       legend = c("h = 0.15", "h = 0.50", "h = 2.00"),
       col    = c("firebrick", "steelblue", "darkgreen"),
       lwd = 2, lty = c(2, 1, 3), bty = "n", cex = 0.8)
```



(c) Explain the bias–variance trade-off for the bandwidth h. What happens when h is too small or too large?

(d) Two principled methods for choosing h are **leave-one-out (LOO) cross-validation** and **data splitting**. Briefly explain the idea behind each and state what criterion is minimised in both cases.

Solution

(a) A valid kernel K must satisfy:

1. **Non-negativity**: $K(u) \geq 0$ for all u.
2. **Symmetry**: $K(-u) = K(u)$ for all u.
3. **Unit integral**: $\int_{-\infty}^{\infty} K(u)\,du = 1$.

For the Gaussian kernel $K(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$:

1. $e^{-u^2/2} > 0$ for all u, so $K(u) > 0$. ✓
2. $K(-u) = \frac{1}{\sqrt{2\pi}} e^{-(-u)^2/2} = \frac{1}{\sqrt{2\pi}} e^{-u^2/2} = K(u)$. ✓
3. K is the standard normal density, which integrates to 1. ✓

(b) $h = 0.15$ (red dashed): very small bandwidth. The KDE is very wiggly with many spurious bumps — it over-fits the data, assigning density too locally near each observation.

$h = 0.50$ (blue solid): moderate bandwidth. The two modes of the mixture are clearly recovered and the estimate is smooth — this is close to the optimal bandwidth for this dataset.

$h = 2.00$ (green dotted): very large bandwidth. The two modes are completely smoothed together into a single broad hump — important features of the true density are lost (under-smoothing in reverse: over-smoothing).

(c) The bandwidth h controls the degree of smoothing:

- h **too small**: each kernel is very narrow, so the estimator is very sensitive to individual observations. This gives low bias but high variance — a wiggly density with spurious modes.
- h **too large**: each kernel spreads probability too diffusely. This gives low variance but high bias — important features such as multimodality are smoothed away.

The optimal h balances these two effects, minimising the mean integrated squared error (MISE) = Bias^2 + Variance.

(d) Both methods minimise an empirical version of the integrated squared error $R(h) = \int (\hat{f}_h(x) - f(x))^2\,dx$.

LOO cross-validation: for each candidate h, the criterion is

$$\widehat{R}_{\text{LOO}}(h) = \int \hat{f}_h^2(x)\,dx - \frac{2}{n}\sum_{i=1}^{n} \hat{f}_{h,-i}(X_i),$$

where $\hat{f}_{h,-i}$ is the KDE with observation X_i removed. The idea is to evaluate how well the density estimated from all other points predicts the held-out point X_i, avoiding using the same observation both to construct and to evaluate the estimator.

Data splitting: split the data into a training set Y and a validation set Z of equal size n. For each candidate h_j, compute a KDE $\hat{f}_{h_j}$ on Y and evaluate its risk on Z:

$$\widehat{L}_j = \int \hat{f}_{h_j}^2(x)\,dx - \frac{2}{n}\sum_{i=1}^{n} \hat{f}_{h_j}(Z_i).$$

Choose the h_j that minimises $\widehat{L}_j$. The idea is the same as LOO but uses an independent split rather than leave-one-out.

Exercise 3 — Mercer Kernels and the Kernel Trick

(a) State the two conditions a function $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ must satisfy to be a valid (Mercer) kernel, and explain what the **Gram matrix** is.

(b) Consider the degree-2 polynomial kernel $K(\mathbf{x}, \mathbf{z}) = \langle \mathbf{x}, \mathbf{z} \rangle^2$ for $\mathbf{x} = (x_1, x_2)^\top$ and $\mathbf{z} = (z_1, z_2)^\top$.

- Expand $K(\mathbf{x}, \mathbf{z})$ algebraically.
- Identify the explicit feature map $\phi(\mathbf{x})$ such that $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$.
- What is the dimension of the feature space?

(c) The RBF (Gaussian) kernel is

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\ell^2} \right).$$

Describe the effect of the length-scale ℓ on which pairs of points the kernel considers similar. What is unusual about the feature space that this kernel corresponds to?

(d) Explain the **kernel trick** in one or two sentences: what computational advantage does it provide, and in which part of an algorithm does it apply?

Solution

(a) A function K is a valid Mercer kernel if it is:

1. **Symmetric**: $K(\mathbf{x}, \mathbf{x}') = K(\mathbf{x}', \mathbf{x})$ for all $\mathbf{x}, \mathbf{x}'$.
2. **Positive semi-definite (PSD)**: for any N points $\mathbf{x}_1, \ldots, \mathbf{x}_N$ and any $\mathbf{c} \in \mathbb{R}^N$, $\sum_{i,j} c_i c_j K(\mathbf{x}_i, \mathbf{x}_j) \geq 0$, equivalently the Gram matrix $\mathbf{K}_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ is PSD.

The **Gram matrix** is the $N \times N$ matrix of all pairwise kernel evaluations on the training inputs: $\mathbf{K}_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$.

(b) Expanding:

$$K(\mathbf{x}, \mathbf{z}) = (x_1 z_1 + x_2 z_2)^2 = x_1^2 z_1^2 + 2 x_1 x_2 z_1 z_2 + x_2^2 z_2^2.$$

The feature map is

$$\phi(\mathbf{x}) = \left(x_1^2,\ \sqrt{2}\, x_1 x_2,\ x_2^2 \right)^\top,$$

since $\phi(\mathbf{x})^\top \phi(\mathbf{z}) = x_1^2 z_1^2 + 2 x_1 x_2 z_1 z_2 + x_2^2 z_2^2 = K(\mathbf{x}, \mathbf{z})$.

The feature space has dimension **3**, even though the original input space has dimension 2. A degree-d polynomial kernel over p-dimensional inputs maps to a feature space of dimension $\binom{p+d}{d}$, which grows rapidly with p and d.

(c) - **Small** ℓ: only very close points in input space receive high kernel values; the kernel is very local. - **Large** ℓ: points can be far apart and still be considered similar; the kernel is very global.

The RBF kernel corresponds to an implicit feature map into an **infinite-dimensional** Hilbert space. Despite this, the kernel itself is computed with a simple closed-form formula, so the kernel trick avoids ever constructing this infinite-dimensional representation explicitly.

(d) The kernel trick replaces every inner product $\langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$ in an algorithm with the kernel evaluation $K(\mathbf{x}, \mathbf{z})$, which can be computed directly without constructing ϕ. This is useful in the dual formulation of learning algorithms (such as SVM), where the optimization and prediction depend on the data only through pairwise inner products.

Exercise 4 — Gaussian Process Regression

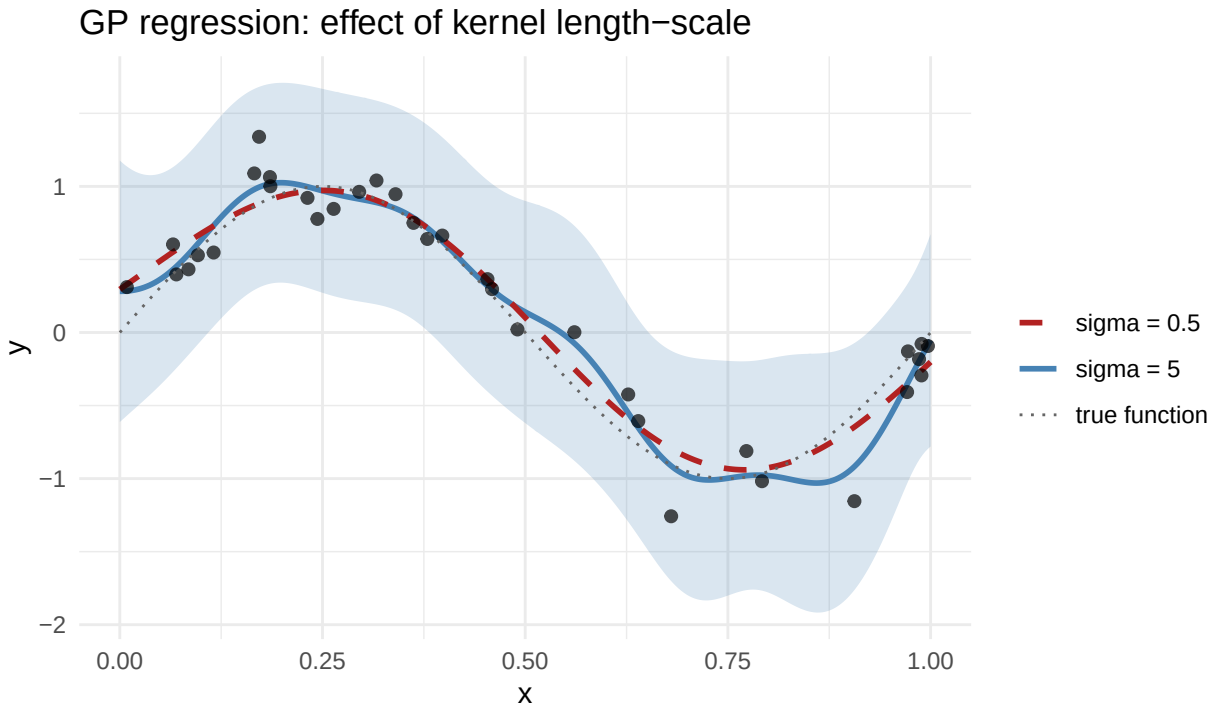
(a) A Gaussian process is written $f \sim \mathcal{GP}(m, \mathcal{K})$. Explain the role of the mean function $m(\mathbf{x})$ and the covariance (kernel) function $\mathcal{K}(\mathbf{x}, \mathbf{x}')$. What distribution does $\big(f(\mathbf{x}_1), \ldots, f(\mathbf{x}_n)\big)$ follow for any finite set of inputs?

(b) In **noisy** GP regression, observations are $y_i = f(\mathbf{x}_i) + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$. Write down the covariance matrix of the observed vector $\mathbf{y} = (y_1, \ldots, y_n)^\top$ and explain why a noise-free GP would not be appropriate for most real datasets.

(c) The code below fits a GP regression model to noisy data using `gausspr()` from `kernlab`. Run the code and then answer the questions that follow.

```
library(kernlab)

set.seed(7)
n       <- 35
x_train <- sort(runif(n, 0, 1))
y_train <- sin(2 * pi * x_train) + rnorm(n, sd = 0.25)
x_grid  <- seq(0, 1, length.out = 300)

# Fit GP with RBF kernel
gp1 <- gausspr(x_train, y_train,
               kernel = "rbfdot",
               kpar   = list(sigma = 5),
               var    = 0.09,
               variance.model = TRUE)

gp2 <- gausspr(x_train, y_train,
               kernel = "rbfdot",
               kpar   = list(sigma = 0.5),
               var    = 0.09,
               variance.model = TRUE)

mu1 <- as.vector(predict(gp1, x_grid))
sd1 <- as.vector(predict(gp1, x_grid, type = "sdeviation"))
mu2 <- as.vector(predict(gp2, x_grid))

df_tr  <- data.frame(x = x_train, y = y_train)
df_grd <- data.frame(x  = x_grid,
                     mu1 = mu1,
                     lo1 = mu1 - 1.96 * sd1,
                     hi1 = mu1 + 1.96 * sd1,
                     mu2 = mu2)

ggplot(df_grd, aes(x = x)) +
  geom_ribbon(aes(ymin = lo1, ymax = hi1),
              fill = "steelblue", alpha = 0.2) +
  geom_line(aes(y = mu1, colour = "sigma = 5"),   linewidth = 1) +
  geom_line(aes(y = mu2, colour = "sigma = 0.5"), linewidth = 1,
            linetype = "dashed") +
  geom_point(data = df_tr, aes(x, y),
             colour = "black", size = 1.8, alpha = 0.7) +
  geom_line(aes(y = sin(2 * pi * x), colour = "true function"),
            linetype = "dotted") +
  scale_colour_manual(values = c("sigma = 5"     = "steelblue",
```

```
                                "sigma = 0.5"    = "firebrick",
                                "true function" = "grey40")) +
labs(title = "GP regression: effect of kernel length-scale",
     x = "x", y = "y", colour = "") +
theme_minimal()
```



- Which model (`sigma = 5` or `sigma = 0.5`) fits more closely to the data and which is smoother? Explain why.
- Where is the posterior uncertainty (shaded band) largest, and what does this reflect about the GP?
- The true function is $\sin(2\pi x)$ with noise SD $= 0.25$, so the noise variance is 0.09 (`var = 0.09`). What would happen if you set `var = 0.001`?

Solution

(a) The **mean function** $m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$ encodes our prior belief about the average behaviour of the function; it is often set to zero when no prior knowledge is available.

The **covariance function** $\mathcal{K}(\mathbf{x}, \mathbf{x}')$ encodes how strongly function values at $\mathbf{x}$ and $\mathbf{x}'$ co-vary. A large value means the function is expected to be similar at those two inputs; a small value means they are nearly independent. The kernel thus controls the smoothness and scale of functions drawn from the GP.

For any finite collection of inputs, the GP assumption implies

$$\big(f(\mathbf{x}_1), \ldots, f(\mathbf{x}_n)\big) \sim \mathcal{N}\big(\mathbf{m}_X,\ K_{X,X}\big),$$

where $\mathbf{m}_X = (m(\mathbf{x}_1), \ldots, m(\mathbf{x}_n))^\top$ and $(K_{X,X})_{ij} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$.

(b) With i.i.d. Gaussian noise $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ independent of f:

$$\mathrm{Cov}(\mathbf{y}) = K_{X,X} + \sigma^2 I.$$

A noise-free GP would require the posterior to pass exactly through every observation. Real data always contain measurement error, so interpolating exactly through every point over-fits the noise and leads to poor generalisation. Adding $\sigma^2 I$ to the covariance allows the GP to smooth over the noise rather than interpolate it.

(c)

Length-scale effect: In `kernlab`'s `rbfdot`, the kernel is $K(x, x') = \exp(-\sigma\|x - x'\|^2)$. A **larger `sigma`** (here 5) corresponds to a **shorter** length-scale: only nearby points are considered similar, so the posterior mean can follow rapid changes in the data and fits more closely. A **smaller `sigma`** (here 0.5) corresponds to a **longer** length-scale: points far apart are still correlated, producing a very smooth, slowly varying posterior mean that may under-fit.

Posterior uncertainty: The shaded band (posterior mean ± 1.96 posterior SD) is widest in regions with few nearby training points (e.g. near the boundary) and narrowest near the training observations. This reflects the fundamental GP property: conditioning on data reduces uncertainty near the observed inputs, while regions without data retain near-prior uncertainty.

Effect of `var = 0.001`: Setting the noise variance to near zero tells the GP the observations are essentially noiseless, forcing the posterior mean to interpolate through every training point. This produces a very wiggly fit that over-fits the noise rather than recovering the smooth underlying $\sin(2\pi x)$ function.

Exercise 5 — Support Vector Machines

(a) The maximal margin classifier solves

$$\min_{\beta_0, \boldsymbol{\beta}} \frac{1}{2}\|\boldsymbol{\beta}\|^2 \quad \text{subject to} \quad y_i(\mathbf{x}_i^\top \boldsymbol{\beta} + \beta_0) \geq 1, \quad i = 1, \ldots, n.$$

Explain what the **margin** is and why we want to maximise it. What are **support vectors** and which KKT condition identifies them?

(b) The support vector classifier (SVC) introduces slack variables $\epsilon_i \geq 0$ and a cost parameter C:

$$\max_{\beta_0, \boldsymbol{\beta}, \boldsymbol{\epsilon}} M \quad \text{subject to} \quad y_i(\beta_0 + \boldsymbol{\beta}^\top \mathbf{x}_i) \geq M(1 - \epsilon_i), \quad \sum_{i=1}^{n} \epsilon_i \leq C.$$

Describe the role of C. What happens to the number of support vectors and the margin width as C increases?

(c) The SVM extends the SVC to non-linear boundaries using the kernel trick. The dual decision function is

$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^{n} \alpha_i K(\mathbf{x}, \mathbf{x}_i).$$

Explain why we can replace $\langle \phi(\mathbf{x}), \phi(\mathbf{x}_i) \rangle$ with $K(\mathbf{x}, \mathbf{x}_i)$, and give one example of a kernel that implicitly maps to an infinite-dimensional feature space.

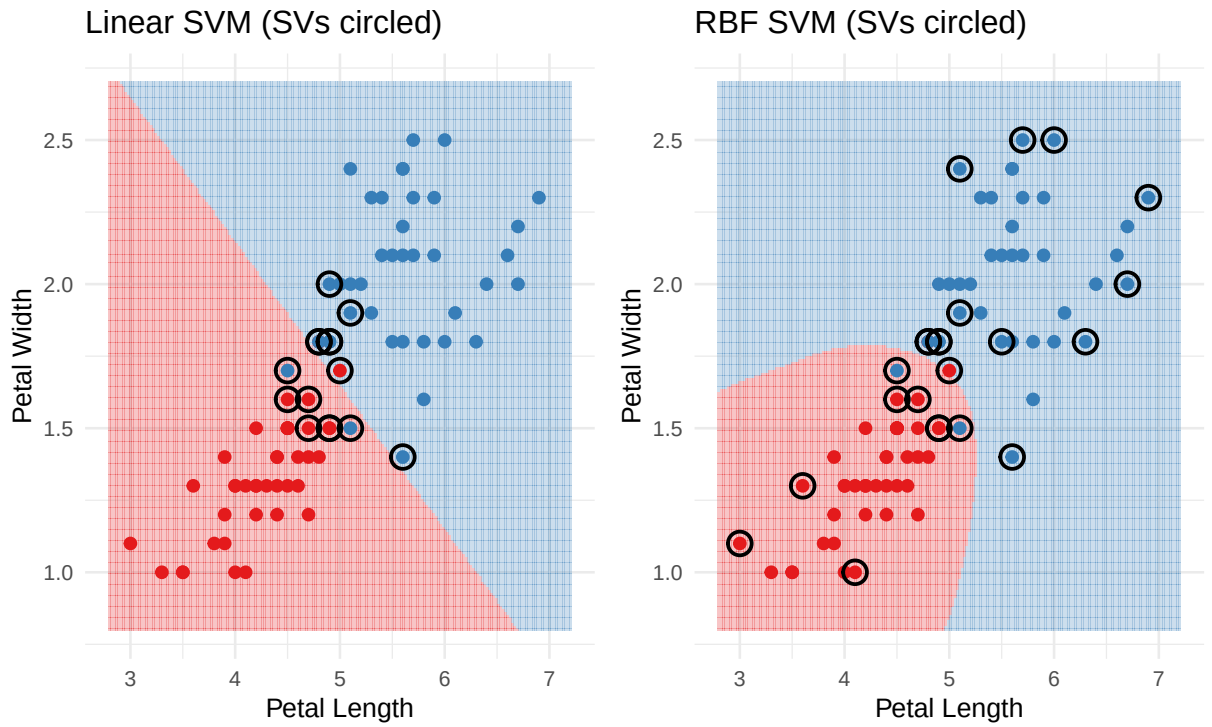
(d) The code below fits linear and RBF kernel SVMs to a two-class subset of the `iris` data and plots the decision boundaries. Run the code and compare the two classifiers in terms of boundary shape, number of support vectors, and test accuracy.

```
library(kernlab)

iris2 <- subset(iris, Species != "setosa")
iris2$Species <- factor(iris2$Species)

set.seed(5)
idx      <- sample(nrow(iris2), 80)
tr       <- iris2[idx, ];   te <- iris2[-idx, ]
```

```
svm_lin <- ksvm(Species ~ Petal.Length + Petal.Width,
                data = tr, kernel = "vanilladot", C = 1)
```

```
##  Setting default kernel parameters
```

```
svm_rbf <- ksvm(Species ~ Petal.Length + Petal.Width,
                data = tr, kernel = "rbfdot",
                kpar = list(sigma = 1), C = 1)

# Prediction grid
g <- expand.grid(
  Petal.Length = seq(2.8, 7.2, length.out = 200),
  Petal.Width  = seq(0.8, 2.7, length.out = 200))
g$lin <- predict(svm_lin, g)
g$rbf <- predict(svm_rbf, g)

sv_lin <- tr[alphaindex(svm_lin)[[1]], ]
sv_rbf <- tr[alphaindex(svm_rbf)[[1]], ]

p_lin <- ggplot() +
  geom_tile(data = g, aes(Petal.Length, Petal.Width, fill = lin),
            alpha = 0.25) +
  geom_point(data = tr,
             aes(Petal.Length, Petal.Width, colour = Species), size = 1.8) +
  geom_point(data = sv_lin,
             aes(Petal.Length, Petal.Width),
             shape = 21, size = 3.5, stroke = 1, colour = "black") +
  scale_fill_brewer(palette = "Set1") +
  scale_colour_brewer(palette = "Set1") +
  labs(title = "Linear SVM (SVs circled)",
       x = "Petal Length", y = "Petal Width") +
  theme_minimal(base_size = 10) + theme(legend.position = "none")

p_rbf <- ggplot() +
  geom_tile(data = g, aes(Petal.Length, Petal.Width, fill = rbf),
            alpha = 0.25) +
  geom_point(data = tr,
             aes(Petal.Length, Petal.Width, colour = Species), size = 1.8) +
  geom_point(data = sv_rbf,
             aes(Petal.Length, Petal.Width),
             shape = 21, size = 3.5, stroke = 1, colour = "black") +
  scale_fill_brewer(palette = "Set1") +
  scale_colour_brewer(palette = "Set1") +
  labs(title = "RBF SVM (SVs circled)",
       x = "Petal Length", y = "Petal Width") +
  theme_minimal(base_size = 10) + theme(legend.position = "none")

library(patchwork)
p_lin + p_rbf
```




```
# Test accuracy
cat("Linear SVM test accuracy:",
    round(mean(predict(svm_lin, te) == te$Species), 3), "\n")
```

```
## Linear SVM test accuracy: 0.95
```

```
cat("RBF SVM test accuracy:   ",
    round(mean(predict(svm_rbf, te) == te$Species), 3), "\n")
```

```
## RBF SVM test accuracy:    0.95
```

```
cat("Linear SVM support vectors:", nSV(svm_lin), "\n")
```

```
## Linear SVM support vectors: 15
```

```
cat("RBF SVM support vectors:   ", nSV(svm_rbf), "\n")
```

```
## RBF SVM support vectors:    23
```

(e) SVM is naturally a binary classifier. Describe the two standard strategies (**one-vs-one** and **one-vs-all**) for extending SVM to $K > 2$ classes, and state when each is recommended.

Solution

(a) The **margin** is the perpendicular distance from the decision hyperplane $\mathbf{x}^\top \boldsymbol{\beta} + \beta_0 = 0$ to the nearest training observation. Maximising the margin provides the largest possible buffer zone between the two classes, which improves generalisation to new data.

Support vectors are the training observations that lie exactly on the margin boundary ($y_i f(\mathbf{x}_i) = 1$). From the KKT condition $\alpha_i(y_i f(\mathbf{x}_i) - 1) = 0$, only observations with $y_i f(\mathbf{x}_i) = 1$ can have $\alpha_i > 0$; all others have $\alpha_i = 0$ and do not contribute to the decision function. The boundary is thus determined entirely by the support vectors.

(b) The parameter C controls the penalty on margin violations:

- **Large** C: violations are heavily penalised; the classifier tries hard to classify all training points correctly, leading to a **narrower margin** and fewer support vectors. Risk of over-fitting increases.
- **Small** C: violations are tolerated; the classifier accepts more misclassifications in exchange for a **wider margin**, resulting in more support vectors. The boundary is smoother and more regularised.

In practice C is tuned by cross-validation.

(c) In the dual formulation, the optimisation and prediction depend on the data only through inner products $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$. After feature mapping ϕ, these become $\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$. A valid Mercer kernel satisfies $K(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$ for some ϕ, so we can substitute K for every inner product without ever computing ϕ explicitly. This is the kernel trick.

The **RBF / Gaussian kernel** $K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$ corresponds to an infinite-dimensional feature map; the kernel allows us to work in this space at the cost of a single scalar evaluation.

(d) Boundary shape: the linear SVM produces a straight-line boundary in the $(Petal.Length, Petal.Width)$ plane, while the RBF SVM can produce a curved boundary. For this near-linearly-separable subset of iris, the difference in boundary shape is subtle but visible.

Support vectors: the linear SVM typically selects fewer support vectors (the data are nearly linearly separable); the RBF SVM may select more, depending on σ and C.

Test accuracy: both kernels tend to achieve high accuracy (often $\geq 90\%$) on this well-separated two-class problem. Any difference highlights the gain (or cost) of the additional RBF flexibility.

(e) One-vs-one: fit $\binom{K}{2}$ binary classifiers, one for each pair of classes. Classify a new point by having each classifier cast a vote, and assign the class with the most votes. Recommended when K is small.

One-vs-all: fit K binary classifiers, each separating one class from all others. For a new point, compute the signed distance to each boundary and assign the class with the largest value. Recommended when K is large, since it requires only K classifiers instead of $\binom{K}{2}$.